

Guía de Práctica

Motor DC con PID conectado a InfluxDB y Grafana

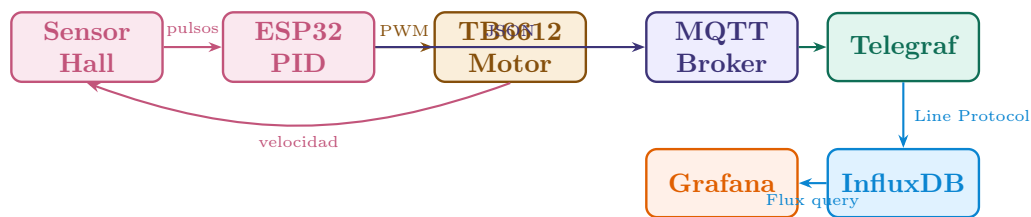
Materia de Verano: IoT · Ingeniería Mecatrónica · 6.º semestre · Universidad Panamericana

Alumno 1: Eduardo Hernández Torres **Alumno 2:** Carlo Gordillo Arenas

Alumno 3: Alejandro Acosta Salazar

Equipo #: 2

Fecha: July 21, 2026



Objetivo: Agregar publicación MQTT al código PID que ya funciona, sin modificar la lógica de control. Al terminar, el dashboard de Grafana muestra en tiempo real: RPM, setpoint, error, señal de control (PWM) y todos los cambios que hacen desde la laptop.

Hardware: ESP32 + Sensor Hall + TB6612FNG + Motor DC

Software en laptop: Docker con stack de la práctica anterior

Contents

1	¿Qué vas a agregar al código PID?	3
2	Código completo — PID + MQTT	3
2.1	platformio.ini — agregar las librerías	3
2.2	main.cpp — PID con publicación MQTT	4
3	Entender el código antes de flashear	9
4	Modificar los parámetros antes de flashear	10
5	Verificar que los datos llegan a InfluxDB	12
6	Actualizar el dashboard de Grafana	12
7	Cambiar el setpoint desde Grafana	14
8	Experimento: observar el PID en acción desde Grafana	15
9	Análisis de resultados	15
10	Troubleshooting	16

1. ¿Qué vas a agregar al código PID?

Tu PID ya funciona — el motor gira y sigue el setpoint. Lo que falta es que esos datos viajen a InfluxDB para poder verlos en Grafana. El cambio es mínimo: **solo se agrega la publicación MQTT, sin tocar la lógica del PID.**

Concepto clave — ¿Por qué no escribir directo a InfluxDB desde el ESP32?

El ESP32 podría escribir directo a la API de InfluxDB por HTTP, pero hay un problema: si InfluxDB se reinicia o falla por un momento, los datos se pierden. Con MQTT el broker actúa como amortiguador — guarda los mensajes hasta que Telegraf los procesa. El ESP32 solo necesita saber la IP del broker, no la de InfluxDB. Esto se llama **desacoplamiento** y es una buena práctica de arquitectura IoT.

Los tres cambios al código son:

1. **Agregar WiFi y MQTT** — nuevas librerías y conexión
2. **Publicar telemetría** — un JSON con RPM, setpoint, error y PWM cada vez que el PID calcula un nuevo ciclo
3. **Recibir setpoint por MQTT** — el operador puede cambiar el setpoint desde Grafana sin tocar el ESP32

Pregunta de reflexión — Antes de empezar

¿Qué datos del motor son útiles monitorear remotamente? Escribe al menos 4 variables y para qué sirve ver cada una:

Variable	¿Para qué sirve verla en el dashboard?
RPM actual	Para saber que está haciendo el sistema en cada momento.
Setpoint	Para saber que debería de estar haciendo el sistema o que quiero que haga.
Error (SP - RPM)	Para saber que tan bien o tan mal está trabajando el control.
Señal PWM	Para saber que tan cargado está el motor.

2. Código completo — PID + MQTT

△ Importante:

Este código reemplaza tu `main.cpp` completo. Antes de hacerlo: copia tu código actual a un archivo `main_pid_original.cpp` como respaldo. Los parámetros `Kp`, `Ki`, `Kd` que ya tienes ajustados van en las líneas marcadas con `// <- TU VALOR`.

2.1. platformio.ini — agregar las librerías

```
1 [env: esp32dev]
```

```

2 platform      = espressif32
3 board         = esp32dev
4 framework     = arduino
5 monitor_speed = 115200
6
7 lib_deps =
8     knolleary/PubSubClient      ; Cliente MQTT
9     bblanchon/ArduinoJson      ; Serializar el payload JSON

```

Listing 1: platformio.ini actualizado

2.2. main.cpp — PID con publicación MQTT

```

1 // =====
2 // Motor DC con PID + publicacion MQTT a InfluxDB/Grafana
3 // Universidad Panamericana - IoT - Semana 2
4 //
5 // Hardware:
6 //   Motor PWM -> GPIO 25 (LEDC canal 0)
7 //   TB6612 IN1 -> GPIO 26
8 //   TB6612 IN2 -> GPIO 27
9 //   Sensor Hall-> GPIO 34 (interrupcion RISING)
10 // =====
11
12 #include <Arduino.h>
13 #include <WiFi.h>
14 #include <PubSubClient.h>
15 #include <ArduinoJson.h>
16
17 //      WiFi
18
19 const char* WIFI_SSID      = "TU_RED_WIFI";           // <-- CAMBIAR
20 const char* WIFI_PASSWORD = "TU_CONTRASENIA";        // <-- CAMBIAR
21
22 //      MQTT (IP de la laptop con Docker)
23
24 const char* MQTT_BROKER = "192.168.1.100";           // <-- CAMBIAR
25 const int   MQTT_PORT   = 1883;
26
27 const char* TOPIC_TELEMETRIA = "iot/motor/telemetry";
28 const char* TOPIC_SETPPOINT  = "iot/motor/setpoint";
29
30 //      Pines
31
32 #define PIN_PWM  25
33 #define PIN_IN1  26
34 #define PIN_IN2  27
35 #define PIN_HALL 34
36
37 //      LEDC (PWM del ESP32)

```

```

35 #define LEDC_CH    0
36 #define LEDC_FREQ  20000
37 #define LEDC_RES    8          // 0-255
38
39 //          Sensor Hall
40
41 #define PPR 2                // Pulsos Por Revolucion de tu sensor
42                             // <-- CAMBIAR segun tu hardware
43
44 //          Parametros PID
45
46 // Usa los valores que ya tienes ajustados de la practica anterior
47 float Kp = 1.0;              // <-- TU VALOR
48 float Ki = 0.5;              // <-- TU VALOR
49 float Kd = 0.1;              // <-- TU VALOR
50
51 //          Variables del PID
52
53 float setpoint      = 300.0; // RPM objetivo inicial
54 float rpm            = 0.0;
55 float errorPrev     = 0.0;
56 float integral      = 0.0;
57 float pwmSalida     = 0.0;
58
59 //          Variables del sensor Hall
60
61 volatile uint32_t contadorPulsos = 0;
62 portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
63
64 //          Temporizadores
65
66 #define T_PID_MS    10        // Loop del PID cada 10 ms (100 Hz)
67 #define T_MQTT_MS   500       // Publicar a MQTT cada 500 ms
68 uint32_t tPID = 0;
69 uint32_t tMQTT = 0;
70
71 //          MQTT
72
73 WiFiClient  wifiClient;
74 PubSubClient mqttClient(wifiClient);
75 bool        mqttListo = false;
76
77 //          ISR: cuenta pulsos del sensor Hall
78
79 void IRAM_ATTR hallISR() {
80     portENTER_CRITICAL_ISR(&mux);
81     contadorPulsos++;
82     portEXIT_CRITICAL_ISR(&mux);

```

```

76 }
77
78 //          Calcular RPM
79
80 float calcularRPM(uint32_t dt_ms) {
81     portENTER_CRITICAL(&mux);
82     uint32_t pulsos = contadorPulsos;
83     contadorPulsos = 0;
84     portEXIT_CRITICAL(&mux);
85     // RPM = (pulsos / PPR) * (60000 / dt_ms)
86     return ((float)pulsos / PPR) * (60000.0f / dt_ms);
87 }
88
89 //          Aplicar PWM al motor (con direccion)
90
91 void aplicarPWM(float valor) {
92     valor = constrain(valor, -255.0f, 255.0f);
93     if (valor >= 0) {
94         digitalWrite(PIN_IN1, HIGH);
95         digitalWrite(PIN_IN2, LOW);
96         ledcWrite(LEDC_CH, (int)valor);
97     } else {
98         digitalWrite(PIN_IN1, LOW);
99         digitalWrite(PIN_IN2, HIGH);
100         ledcWrite(LEDC_CH, (int)(-valor));
101     }
102     pwmSalida = valor;
103 }
104
105 //          Calcular un ciclo del PID
106
107 void calcularPID(float dt_s) {
108     float error = setpoint - rpm;
109
110     // Termino proporcional
111     float P = Kp * error;
112
113     // Termino integral con anti-windup
114     integral += error * dt_s;
115     integral = constrain(integral, -200.0f, 200.0f);
116     float I = Ki * integral;
117
118     // Termino derivativo
119     float D = Kd * (error - errorPrev) / dt_s;
120     errorPrev = error;
121
122     // Senial de control
123     float u = P + I + D;
124     aplicarPWM(u);
125 }

```

```

124 //          Conectar WiFi

125 void conectarWiFi() {
126     Serial.printf("Conectando a WiFi '%s'", WIFI_SSID);
127     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
128     int n = 0;
129     while (WiFi.status() != WL_CONNECTED && n++ < 40) {
130         delay(500);
131         Serial.print(".");
132     }
133     if (WiFi.status() == WL_CONNECTED) {
134         Serial.printf("\n OK - IP: %s\n",
135             WiFi.localIP().toString().c_str());
136     } else {
137         Serial.println("\n FALLO WiFi - continuando sin MQTT");
138     }
139 }

140
141 //          Callback MQTT: recibir nuevo setpoint desde Grafana

142 void mqttCallback(char* topic, byte* payload, unsigned int len) {
143     String msg = "";
144     for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];
145
146     if (String(topic) == TOPIC_SETPOINT) {
147         float sp = msg.toFloat();
148         if (sp >= 0 && sp <= 700) {
149             setpoint = sp;
150             integral = 0;    // Reset integral al cambiar setpoint
151             Serial.printf("<- Nuevo setpoint: %.1f RPM\n", setpoint);
152         }
153     }
154 }

155
156 //          Conectar al broker MQTT

157 void conectarMQTT() {
158     if (WiFi.status() != WL_CONNECTED) return;
159     int n = 0;
160     while (!mqttClient.connected() && n++ < 5) {
161         Serial.print("Conectando MQTT...");
162         if (mqttClient.connect("ESP32-Motor-PID")) {
163             mqttClient.subscribe(TOPIC_SETPOINT);
164             mqttListo = true;
165             Serial.println(" OK");
166         } else {
167             Serial.printf(" fallo (rc=%d)\n", mqttClient.state());
168             delay(1000);
169         }
170     }
171 }
172

```

```

173 // Publicar telemetria a MQTT (-> Telegraf -> InfluxDB)

174 void publicarTelemetria() {
175     if (!mqttListo || !mqttClient.connected()) return;
176
177     // Construir el JSON que Telegraf leera
178     // Estos campos se convertiran en Fields en InfluxDB
179     StaticJsonDocument<200> doc;
180     doc["rpm"] = round(rpm * 10.0f) / 10.0f;
181     doc["setpoint"] = setpoint;
182     doc["error"] = round((setpoint - rpm) * 10.0f) / 10.0f;
183     doc["pwm"] = round(pwmSalida);
184     doc["nodo"] = "ESP32-Motor"; // Este sera un Tag en InfluxDB
185
186     char buf[200];
187     serializeJson(doc, buf);
188     mqttClient.publish(TOPIC_TELEMETRIA, buf);
189
190     Serial.printf("-> rpm:%.1f sp:%.1f err:%.1f pwm:%.0f\n",
191                 rpm, setpoint, setpoint - rpm, pwmSalida);
192 }
193
194 // =====
195 void setup() {
196     Serial.begin(115200);
197
198     // Pines del driver TB6612FNG
199     pinMode(PIN_IN1, OUTPUT);
200     pinMode(PIN_IN2, OUTPUT);
201     ledcSetup(LEDC_CH, LEDC_FREQ, LEDC_RES);
202     ledcAttachPin(PIN_PWM, LEDC_CH);
203     aplicarPWM(0);
204
205     // Sensor Hall
206     pinMode(PIN_HALL, INPUT_PULLUP);
207     attachInterrupt(digitalPinToInterrupt(PIN_HALL), hallISR, RISING);
208
209     // WiFi y MQTT (el PID arranca aunque falle la conexion)
210     conectarWiFi();
211     if (WiFi.status() == WL_CONNECTED) {
212         mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
213         mqttClient.setCallback(mqttCallback);
214         mqttClient.setBufferSize(512);
215         conectarMQTT();
216     }
217
218     tPID = millis();
219     tMQTT = millis();
220
221     Serial.println("Sistema listo");
222     Serial.printf("Setpoint inicial: %.1f RPM\n", setpoint);
223 }
224
225 // =====
226 void loop() {

```



```

227 // Mantener conexion MQTT activa
228 if (mqttListo) mqttClient.loop();
229
230 uint32_t ahora = millis();
231
232 //          Loop del PID (cada T_PID_MS)
233
234 if ((ahora - tPID) >= T_PID_MS) {
235     float dt_s = (ahora - tPID) / 1000.0f;
236     rpm        = calcularRPM(ahora - tPID);
237     calcularPID(dt_s);
238     tPID = ahora;
239 }
240
241 //          Publicar a MQTT (cada T_MQTT_MS)
242
243 if ((ahora - tMQTT) >= T_MQTT_MS) {
244     publicarTelemetry();
245     tMQTT = ahora;
246
247     // Reconectar MQTT si se cayo
248     if (mqttListo && !mqttClient.connected()) {
249         conectarMQTT();
250     }
251 }

```

Listing 2: main.cpp completo — PID + MQTT + InfluxDB

3. Entender el código antes de flashear

Concepto clave — ¿Por qué el PID corre cada 10 ms y MQTT cada 500 ms?

El PID necesita un intervalo de tiempo fijo y corto para calcular bien el término derivativo ($K_d \cdot \Delta e / \Delta t$). Si el intervalo varía, la derivada se distorsiona y el control empeora. MQTT en cambio no necesita ser tan rápido — 2 publicaciones por segundo son más que suficientes para ver los datos en Grafana. Los dos procesos corren en el mismo `loop()` pero con temporizadores independientes.

Concepto clave — ¿Qué es el anti-windup?

Cuando el motor no puede alcanzar el setpoint (por ejemplo, si pides 700 RPM pero el motor solo llega a 600), el término integral sigue acumulando error. Si dejas que crezca sin límite, cuando el setpoint baje a 300 el integral acumulado sigue empujando el motor hacia arriba durante varios segundos — el motor *se pasa* del objetivo. El `constrain(integral, -200, 200)` evita eso. En Grafana verán ese fenómeno como un sobreimpulso prolongado si lo desactivan.

Concepto clave — ¿Qué viaja por MQTT y cómo llega a InfluxDB?

El ESP32 publica este JSON en el topic `iot/motor/telemetry`:

```
{ "rpm": 298.5, "setpoint": 300.0, "error": 1.5, "pwm": 145, "nodo": "ESP32-Motor" }
```

Telegraf está suscrito a ese topic. Cuando llega el mensaje:

1. Lee el JSON
2. Lo convierte a Line Protocol de InfluxDB:

```
mqtt_consumer,nodo=ESP32-Motor rpm=298.5,setpoint=300.0,error=1.5,pwm=145 1705312800000000000
```

Nota cómo **nodo** se convierte en **Tag** (metadata) y los valores numéricos en **Fields** (datos que cambian). Esto ocurre automáticamente — no escriben una línea de código de integración.

Pregunta de reflexión — Antes de flashear

Responde estas preguntas con base en el código que acabas de leer:

1. ¿Qué pasa con el PID si el WiFi no está disponible? ¿El motor sigue controlado? Debería de seguir controlando pues el PID se corre dentro del microcontrolador solamente no se podría mandar a la nube por lo que no lo podríamos ver en el dashboard.
2. ¿En qué línea se resetea el integral cuando llega un nuevo setpoint por MQTT? ¿Por qué es importante hacer ese reset? En la 150, es importante porque se acumularía el error a través de múltiples setpoints lo cual no nos serviría de nada y solamente provocaría fallas en el control.
3. ¿Cuántas veces por segundo se publica telemetría a MQTT con la configuración actual? 2 veces por segundo.

4. Modificar los parámetros antes de flashear

Paso 1 — Cambiar las tres líneas obligatorias

Abrir `main.cpp` en VS Code y cambiar:

```
const char* WIFI_SSID      = "TU_RED_WIFI";           // Red del salon
const char* WIFI_PASSWORD = "TU_CONTRASENIA";        // Contraseña
const char* MQTT_BROKER    = "192.168.1.X";           // IP de tu laptop
```

△ Importante:

Para encontrar la IP de tu laptop:

Windows: `ipconfig` → Dirección IPv4 del adaptador WiFi

Mac: `ipconfig getifaddr en0`

Linux: `hostname -I`

El ESP32 y la laptop deben estar en la misma red WiFi.

Paso 2 — Poner tus parámetros PID

Sustituir los valores de Kp, Ki, Kd con los que ya tienes ajustados:

```
float Kp = 1.0;    // <-- TU VALOR de la practica anterior
float Ki = 0.5;    // <-- TU VALOR
float Kd = 0.1;    // <-- TU VALOR
```

Paso 3 — Verificar PPR del sensor Hall

PPR (Pulsos Por Revolución) debe coincidir con tu sensor. Si no lo sabes, gira el eje del motor a mano lentamente y cuenta cuántos pulsos aparecen en el Serial Monitor por vuelta completa.

```
#define PPR 2      // <-- Cambiar si tu sensor da diferente numero de pulsos
```

Paso 4 — Verificar pines del TB6612FNG

Confirmar que los pines GPIO corresponden a tu conexión física:

Pin TB6612	GPIO ESP32	Descripción
PWMA	25	Señal PWM de velocidad
AIN1	26	Dirección bit 1
AIN2	27	Dirección bit 2
STBY	3.3V	Habilitar driver (siempre encendido)
VCC	3.3V	Alimentación lógica
VM	Fuente motor	Alimentación del motor (7-12V)
GND	GND	Tierra común
Hall signal	34	Pulsos del sensor Hall
Hall VCC	3.3V	Alimentación del sensor

Table 1: Conexiones ESP32 → TB6612FNG + Sensor Hall

Paso 5 — Compilar y flashear

1. Clic en **PlatformIO: Build** y esperar que compile sin errores
2. Conectar el ESP32 por USB
3. Clic en **PlatformIO: Upload**
4. Abrir el Serial Monitor (115200 baud)

✓ Verifica antes de continuar:

El Serial Monitor debe mostrar esta secuencia:

```
Conectando a WiFi 'MiRed'..... OK - IP: 192.168.1.105
Conectando MQTT... OK
Sistema listo
```

```
Setpoint inicial: 300.0 RPM
-> rpm:0.0   sp:300.0   err:300.0   pwm:255
-> rpm:145.3 sp:300.0   err:154.7   pwm:230
-> rpm:287.6 sp:300.0   err:12.4    pwm:168
-> rpm:298.9 sp:300.0   err:1.1     pwm:152
```

Si no aparece la línea de WiFi: verificar SSID y contraseña.

Si WiFi conecta pero MQTT falla: verificar la IP del broker y que Docker esté corriendo.

5. Verificar que los datos llegan a InfluxDB

Paso 6 — Abrir InfluxDB y verificar los datos

1. Abrir <http://localhost:8086>
2. Ir a **Data Explorer**
3. Cambiar a modo **Script Editor**
4. Ejecutar esta consulta:

```
from(bucket: "iot-verano")
  |> range(start: -2m)
  |> filter(fn: (r) => r["_field"] == "rpm")
  |> yield(name: "rpm_motor")
```

Listing 3: Consulta para ver los datos del motor real

✓ Verifica antes de continuar:

Deben aparecer puntos de datos con valores reales de RPM. Si no aparece nada después de 30 segundos revisar los logs de Telegraf con:

```
docker logs iot-telegraf --tail 20
```

Concepto clave — ¿Qué ves en InfluxDB en este momento?

Cada punto en la gráfica de InfluxDB representa una medición que viajó así: ESP32 → JSON → MQTT → Telegraf → Line Protocol → InfluxDB. El tiempo de tránsito total es aproximadamente 1-2 segundos. Por eso si cambias el setpoint en Grafana, el motor tarda ~1 segundo en recibir el comando — el PID en el ESP32 responde más rápido que eso, pero la red agrega latencia.

6. Actualizar el dashboard de Grafana

El dashboard que crearon en la práctica anterior ya tiene el panel de RPM funcionando con datos del simulador. Ahora van a agregar paneles para el error del PID y la señal de control (PWM).

Paso 7 — Verificar que el panel de RPM muestra datos reales

1. Abrir <http://localhost:3000>
2. Abrir el dashboard **Motor DC — Semana 2**
3. Cambiar el rango de tiempo a **Last 5 minutes**
4. Verificar que la gráfica muestra las RPM del motor real

Si el panel sigue mostrando datos del simulador: verificar que el simulador ya no está corriendo en ningún ESP32.

Paso 8 — Agregar panel de error del PID

1. En el dashboard: **Add → Visualization**
2. Seleccionar tipo: **Time Series**
3. Pegar esta consulta en modo **Code**:

```
from(bucket: "iot-verano")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_field"] == "error")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "error_pid")
```

Listing 4: Panel — Error del PID en tiempo real

4. **Title:** Error del PID (SP - RPM)
5. **Unit:** RPM
6. En **Thresholds:** agregar línea de referencia en 0
7. Clic en **Apply**

Paso 9 — Agregar panel de señal de control (PWM)

Este panel muestra qué tanta energía le está dando el PID al motor en cada momento.

```
from(bucket: "iot-verano")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_field"] == "pwm")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "pwm_control")
```

Listing 5: Panel — Señal de control PWM

8. **Title:** Señal de control PWM

9. **Unit:** Misc / short (valor 0-255)
10. **Min:** 0 **Max:** 255
11. Clic en **Apply**

Paso 10 — Panel combinado: RPM + Setpoint en la misma gráfica

Este es el panel más útil para ver qué tan bien sigue el PID. Editar el panel de RPM existente y actualizar su consulta:

```
from(bucket: "iot-verano")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_field"] == "rpm" or r["_field"] == "setpoint")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "motor")
```

Listing 6: Panel combinado — RPM y Setpoint superpuestos

✓ Verifica antes de continuar:

El dashboard completo debe mostrar 4 paneles:

- | | |
|--------------------------------|-------------------------------|
| ✓ RPM + Setpoint (Time Series) | ✓ Error del PID (Time Series) |
| ✓ RPM actual (Gauge) | ✓ Señal PWM (Time Series) |

Todos actualizándose solos cada 5 segundos con datos reales del motor.

7. Cambiar el setpoint desde Grafana

Concepto clave — El flujo de un comando de setpoint

Cuando cambias el setpoint desde Grafana, el camino es inverso al de los datos: Grafana publica un mensaje MQTT al topic `iot/motor/setpoint`. El ESP32 está suscrito a ese topic y cuando recibe el mensaje, actualiza la variable `setpoint` del PID en el próximo ciclo de 10 ms.

La latencia total de ese comando es aproximadamente 1-2 segundos (WiFi + MQTT + ciclo del PID). El PID sigue controlando el motor en tiempo real durante esa espera.

Para enviar un setpoint desde la laptop sin usar Grafana todavía, pueden usar el cliente MQTT de línea de comandos dentro de Docker:

```
# Enviar setpoint de 400 RPM al motor
docker exec iot-mosquitto mosquitto_pub \
  -h localhost -p 1883 \
  -t "iot/motor/setpoint" \
  -m "400"
```

Listing 7: Enviar un setpoint desde la terminal

Verificar en el Serial Monitor del ESP32 que aparece: <- Nuevo setpoint: 400.0 RPM

8. Experimento: observar el PID en acción desde Grafana

Paso 11 — Experimento de escalón

Con el motor corriendo y el dashboard visible, cambiar el setpoint varias veces y observar el comportamiento del PID en las gráficas.

1. Ejecutar el siguiente comando para cada setpoint listado en la tabla y esperar 20 segundos antes del siguiente:

```
docker exec iot-mosquitto mosquitto_pub -h localhost -p 1883 -t "iot/motor/setpoint" -m "200"
```

2. Anotar los valores observados en Grafana:

Setpoint (RPM)	RPM estado estacionario	Error residual	PWM estacionario	Observación
40	31 - 47	-7 - 9	46 - 70	Comportamiento esperado
80	70 - 85	-5 - 10	71 - 105	Comportamiento esperado
120	109 - 133	-13 - 11	133 - 164	Comportamiento esperado
170	140 - 164	6 - 30	255	Con la fuente de 12V no alcanza los RPM

Table 2: Respuesta del PID a distintos setpoints — datos de Grafana

9. Análisis de resultados

Pregunta de reflexión — ¿Qué dicen los datos de Grafana sobre tu PID?

- A1.** Observando el panel de error, ¿el motor llega exactamente al setpoint (error = 0) o queda un error residual permanente? ¿Cuál de los términos del PID (P, I o D) corrige ese error residual? Llega en momentos, pero nunca se estabiliza (por el funcionamiento del encoder), y el que corrige el error es la ki.
- A2.** Cuando cambias el setpoint de 40 a 120 RPM de golpe, ¿hay sobreimpulso (el motor pasa momentáneamente de 120)? ¿Qué parámetro del PID podrías reducir para disminuir ese sobreimpulso? Si hay un sobreimpulso de 20 RPM, y se podría reducir la ki.
- A3.** Mirando el panel de PWM durante el estado estacionario a 80 RPM: ¿el PWM es constante o sigue variando? Explica por qué. Sigue variando, ya que lo intenta mantener (como baja poco y sube poco los RPMs, este tiene que compensar).
- A4.** Si desactivaras el término integral ($K_i = 0$) y repitieras el experimento, ¿cómo cambiaría el error residual en estado estacionario? ¿Cómo lo verías en la gráfica de Grafana? Siempre

habría un error residual, pero de manera más notable (no es como que oscilaría entre el setpoint), por lo que no funcionaría.

- A5.** Propón un criterio numérico para decir que tu PID está bien sintonizado, basado en las gráficas de Grafana. Ejemplo: “el error residual debe ser menor a X RPM después de Y segundos”. Escribe el tuyo: Que el error residual sea de ± 8 RPMs cuando ya esté estable (en 0.5 s) y haya pasado el sobreimpulso, ya que es lo que se ve cuando mandas un PWM constante.

10. Troubleshooting

Síntoma	Solución
Motor no gira pero el PID calcula RPM siempre en 0 en Serial Monitor	Verificar STBY del TB6612 en 3.3V. Verificar alimentación VM del motor (7-12V separada del ESP32). Verificar GPIO 34 del sensor Hall. Girar el eje a mano y ver si aparecen pulsos en el monitor. Verificar que el imán está alineado con el sensor.
WiFi conecta pero MQTT falla Datos llegan a MQTT pero no a InfluxDB Grafana muestra “No data”	Verificar IP en MQTT_BROKER. Ejecutar <code>docker compose ps</code> en la laptop — Mosquitto debe estar running. Revisar logs: <code>docker logs iot-telegraf</code> . Verificar que <code>telegraf.conf</code> tiene el topic correcto (<code>iot/motor/#</code>). Cambiar el rango de tiempo a Last 5 minutes . Verificar que el data source tiene URL <code>http://influxdb:8086</code> (no localhost).
El PID oscila y no estabiliza	Reducir Kp a la mitad. Si sigue oscilando, poner $K_i = 0$ y $K_d = 0$ y subir Kp gradualmente.

Lista de verificación final:

- | | |
|--|---|
| ☒ Motor girando con PID activo | ☒ Setpoint cambiado exitosamente por terminal |
| ☒ Serial Monitor muestra RPM reales | ☒ Tabla del experimento de escalón completa |
| ☒ WiFi y MQTT conectados | ☒ 5 preguntas de análisis respondidas |
| ☒ Datos visibles en InfluxDB Data Explorer | ☒ Captura de pantalla del dashboard tomada |
| ☒ Dashboard con 4 paneles actualizándose | ☒ Código subido a GitHub del equipo |

Siguiente paso — Semana 3: Agregar un control de setpoint visual en la plataforma web (FastAPI + Chart.js). El operador moverá un slider en el navegador que publicará al mismo topic MQTT `iot/motor/setpoint`. El ESP32 no cambia — ya está escuchando ese topic.